



操作系统实验报告

Lab 1:The Trouble with Concurrent Programming

专 业： 人工智能

姓 名： 马承乾

学 号： 22920212204180

实验日期： 2023.11.2

指导教师： 曹冬林

目录

1	实验目的	3
1.1	在 Ubuntu20.04 系统上 (64 位) 进行 nachos 环境配置	3
1.2	(实验手册 3.1.1 要求)	3
1.3	(实验手册 3.1.2 要求)	3
2	实验要求	3
2.1	给定的双链表结构 (保存在 dlist.h 中)	3
2.1.1	链表元素声明 :	3
2.1.2	链表类声明	3
2.2	(实验手册 3.1.1 要求)	4
2.2.1	双链表实现	4
2.2.2	代码验证	4
2.3	(实验手册 3.1.2 要求)	4
2.3.1	正确编译	4
2.3.2	修改 threadtest.cc 文件	4
2.3.3	演示并发错误	4
3	实验流程	4
3.1	实验目标	4
3.2	实验准备	5
3.2.1	准备 linux 系统	5
	并且成功解压安装 nachos3.4	5
3.2.2	创建自己的工作目录 ~/lab1	5
3.3	熟悉 nachos	6
3.3.1	nachos 概述	6
3.3.2	目录结构	6
3.3.3	各部分的编译运行	7
3.4	编写并测试双向链表	7
3.4.1	双向链表的定义	7
3.4.2	编写 dlist-driver.cc 函数 , 见附录代码四	7
3.4.3	测试	8
3.5	插入 yield 语句后, 观察并发程序带来的错误	9
3.5.1	Yield 函数和 Sleep 函数	9
4	实验总结	19
4.1	通过环境的配置和前期的准备工作	19
4.2	通过对 thread 目录下各种文件的阅读	19
4.3	学会了更加规范的注释方法	19
4.4	通过对并发程序部分的实验	19
4.5	通过对 makefile 和 gcc 的对比学习	19
5	实验心得	19
6	附录	20

1 实验目的

1.1 在 Ubuntu20.04 系统上（64 位）进行 nachos 环境配置

1.2 （实验手册 3.1.1 要求）

Write a C++ program to implement a doubly -linked list based on the given definitions （基于给定的链表定义用 C++程序实现双链表）

1.3 （实验手册 3.1.2 要求）

Nachos Familiarity and Understanding its Thread System （熟悉 nachos 系统并且理解其线程系统，演示并发错误）

2 实验要求

2.1 给定的双链表结构（保存在 dllist.h 中）

2.1.1 链表元素声明：

```
class DLLElement {
public:
    DLLElement( void *itemPtr, int sortKey );//构造函数
    DLLElement *next;//声明前指针
    DLLElement *prev;//声明后指针
    int key;//声明列表元素的排序 key
    void *item;//声明节点所占的空间
};
```

2.1.2 链表类声明

```
class DLLList {
public:

    DLLList();//构造
    ~DLLList();//析构

    void Prepend(void *item);//声明头插函数
    void Append(void *item);//声明尾插函数
    void *Remove(int *keyPtr);//声明删除函数

    bool IsEmpty();//声明判空函数

    void SortedInsert(void *item, int sortKey);//声明有序插入函数
    void *SortedRemove(int sortKey);//声明有序删除函数

private:
```

```
DLLElement *first;//声明头指针
DLLElement *last;//声明尾指针
};
```

2.2 （实验手册 3.1.1 要求）

2.2.1 双链表实现

双链表应根据整型键对项目进行排序（对于不采用键参数的插入操作，请为该操作指定一致的键值）。编写程序，使其代码包含在三个文件中：dllist.h、dllist.cc 、dllist-driver.cc。

前两个文件应提供上述两个类的定义和实现，第三个文件应包含两个函数，其中一个函数使用随机键生成 N 个项（或者为了帮助调试，可以使用更仔细选择的键顺序控制输入序列），并将其插入到双链接列表中，另一个从列表的开头删除 N 个项目，并将删除的项目打印到控制台。这两个函数都应应将整数 N 和指向列表的指针作为参数。

2.2.2 代码验证

验证是否确实正确实现了类和驱动程序函数，也即创建一个包含程序主函数的单独文件。在此函数中，首先分配链表，然后调用上面的驱动程序函数，为参数传递适当的值，从而验证 dllist-driver.cc 程序正确地实现了双链表。

2.3 （实验手册 3.1.2 要求）

2.3.1 正确编译

将 dllist.h、dllist.cc 和 dllist-driver.cc 文件复制到 threads 子目录中。修改根 nachos 目录中 Makefile.common 中 THREAD_H 和 THREAD_C 的定义，以包含这些文件并更新 Makefile 依赖项，使程序能够用 make 命令正确编译。

2.3.2 修改 threadtest.cc 文件

创建一个类似于文件 threadtest.cc 的驱动程序文件，该文件使用 DLList-driver.cc 中的函数调用 DLList 类对 threads/main.cc 进行更改，以便执行 nachos 命令时会执行新驱动程序文件中的函数，而不是文件 threadtest.cc 中的函数 threadtest。

2.3.3 演示并发错误

修改 threadtest.cc 和 dllist.cc 等程序，演示程序在运行过程中可能出现的并发错误。

3 实验流程

3.1 实验目标

本次实验目标是通过实现双链表的方式熟悉 nachos 系统，需要修改的文件有 main.cc, threadtest.cc, dllist-driver.cc, dllist.cc, dllist.h 等文件，我们通过一张最终程序的运行流程图给出任务完成后各文件之间的关系：

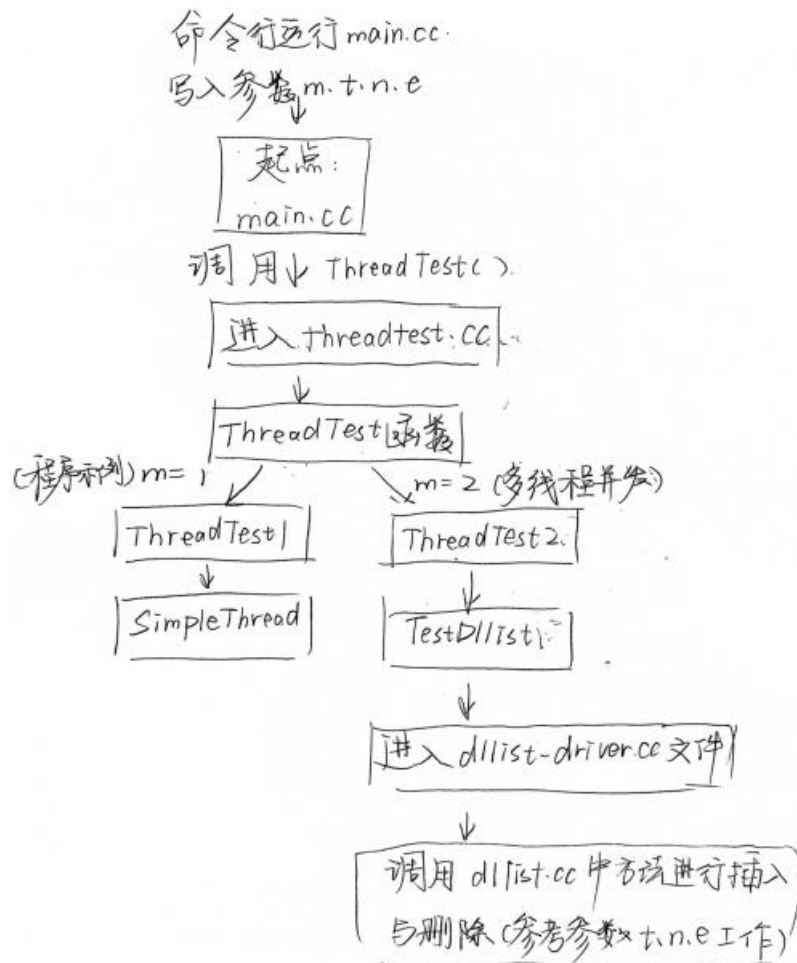


图 0: 最终程序运行流程

3.2 实验准备

3.2.1 准备 linux 系统

并且成功解压安装 nachos3.4

该步骤较为简单，成功后的界面如下：

```

[ai22920212204180@mc0re nachos-3.4]$ cd ..
[ai22920212204180@mc0re ~]$ ls
gnu-decstation-ultrix nachos-3.4
[ai22920212204180@mc0re ~]$
  
```

图 1: nachos 解压成功后包含的两个文件夹（选中部分）

```

[ai22920212204180@mc0re nachos-3.4]$ ls
c++example code COPYRIGHT nachos.ps README
[ai22920212204180@mc0re nachos-3.4]$
  
```

图 2: nachos3.4 文件夹中的内容

3.2.2 创建自己的工作目录~/lab1

将 nachos-3.4 部分的源代码和文件拷贝到其中，这步主要是为了保留 nachos 原始的代码副本

```
[ai22920212204180@mc core code]$ pwd
/home/ai2021/ai22920212204180/lab1/nachos-3.4/code
[ai22920212204180@mc core code]$ ll
总计 88
drwxr-x--- 2 ai22920212204180 caodonglin 4096 11-01 23:54 bin
drwxr-x--- 3 ai22920212204180 caodonglin 4096 11-01 23:54 filesys
drwxr-x--- 2 ai22920212204180 caodonglin 4096 11-01 23:54 machine
-rw-r--r-- 1 ai22920212204180 caodonglin 1134 11-01 23:54 Makefile
-rw-r--r-- 1 ai22920212204180 caodonglin 4070 11-01 23:54 Makefile.common
-rw----- 1 ai22920212204180 caodonglin 744 11-01 23:54 Makefile.dep
drwxr-x--- 2 ai22920212204180 caodonglin 4096 11-01 23:54 network
drwxr-x--- 2 ai22920212204180 caodonglin 4096 11-01 23:54 test
drwxr-x--- 2 ai22920212204180 caodonglin 4096 11-01 23:54 threads
drwxr-x--- 2 ai22920212204180 caodonglin 4096 11-01 23:54 userprog
drwxr-x--- 2 ai22920212204180 caodonglin 4096 11-01 23:54 vm
[ai22920212204180@mc core code]$
```

图 3：工作环境目录

配置好的工作环境如上所示，接下来就可以进行开发。

3.3 熟悉 nachos

3.3.1 nachos 概述

Nachos 建立在一个软件模拟的虚拟机上，主要包括机器模拟、线程管理、文件系统管理、用户程序和虚拟存储以及网络系统。

3.3.2 目录结构

对图二中的目录结构做 以下说明：

目录中含有：

copyright	文件	Nachos 的版权信息
readme	文件	Nachos 的 readme 信息
nachos.ps	文件	Nachos 的介绍文档 (Postscript 格式)
c++example	目录	有关 C++介绍和实例
code	目录	Nachos 各个部分的源代码

最主要的部分是 Nachos 的源代码部分。它的目录结构是：

Makefile	文件	Nachos 的 Makefile 文件。
Makefile.common	文件	
Makefile.dep	文件	当 Nachos 需要移植到其它系统时，可以修改
Makefile.dep 中的 HOST 参数		
machine	目录	Nachos 虚拟机模拟部分源代码
threads	目录	Nachos 线程管理部分源代码
fileysys	目录	Nachos 文件系统管理部分源代码
userprog	目录	Nachos 用户程序部分源代码
network	目录	Nachos 网络管理部分源代码
vm	目录	Nachos 虚拟内存管理部分源代码
test	目录	一些测试用应用程序

bin 目录 包含有用户程序目标码变换的程序

3.3.3 各部分的编译运行

Nachos 的各个部分都可以独立编译运行，也可以同时编译各个部分。全部编译可以采用如下命令：

代码一：

```
[ai22920212204180@mcore code]$ make
```

当需要单独编译线程管理部分时，先进入 threads 目录，然后采用如下命令：

代码二：

```
[ai22920212204180@mcore threads]$ make depend
[ai22920212204180@mcore threads]$ make
```

实际上，各部分目录下都有一个 Makefile 文件，内容大体相同，区别在于一些条件编译的参数。比如在单独编译线程管理部分时，文件管理部分就被屏蔽了，这样我们就可以专心于线程管理部分的调试。

注意：在 Linux 文件中做中大型开发时，使用 gcc 命令单独编译会使得工作量比较巨大，通常会编写 makefile 文件来进行一次性编译，本次实验中，我们需要修改 Makefile.common 中 THREAD_H 和 THREAD_C 的定义，以包含我们 自己编写的三个文件，并更新 Makefile 依赖项（也即添加.o 文件）。每次修改完源代码之后都要重新编译才可以运行。

3.4 编写并测试双向链表

3.4.1 双向链表的定义

dllist.cc 实现在 dllist.cc 中，代码见附录中代码三

```
[ai22920212204180@mcore threads]$ ll
总计 692
-rw-r--r-- 1 ai22920212204180 caodonglin 91 11-02 02:50 bool.h
-rw-r--r-- 1 ai22920212204180 caodonglin 1229 11-02 02:50 copyright.h
-rw-r--r-- 1 ai22920212204180 caodonglin 6921 11-02 05:13 dllist.cc
-rw-r--r-- 1 ai22920212204180 caodonglin 956 11-02 05:13 dllist-driver.cc
-rw-r--r-- 1 ai22920212204180 caodonglin 1337 11-02 05:06 dllist.h
-rw-r--r-- 1 ai22920212204180 caodonglin 17208 11-02 02:50 interrupt.o
-rw-r--r-- 1 ai22920212204180 caodonglin 7143 11-02 02:50 list.cc
-rw-r--r-- 1 ai22920212204180 caodonglin 2298 11-02 02:50 list.h
-rw-r--r-- 1 ai22920212204180 caodonglin 7796 11-02 02:50 list.o
-rw-r--r-- 1 ai22920212204180 caodonglin 5619 11-02 05:13 main.cc
-rw-r--r-- 1 ai22920212204180 caodonglin 5132 11-02 02:50 main.o
-rw-r--r-- 1 ai22920212204180 caodonglin 47444 11-02 05:14 Makefile
-rwxr-xr-x 1 ai22920212204180 caodonglin 141209 11-02 02:50 nachos
-rw-r--r-- 1 ai22920212204180 caodonglin 5115 11-02 02:50 scheduler.cc
```

图 4：dllist.cc 函数

3.4.2 编写 dllist-driver.cc 函数，见附录代码四

该函数主要实现对双链表中的元素进行按序插入和删除，代码见附录，这里对重要的函数思路进行讲解：

(1) SortedInsert 方法 void SortedInsert(void *item, int sortKey)

检查链表是否为空。

如果链表为空，则创建一个新的 DLLElement 对象，并将其设置为链表的头部和尾部。

如果链表不为空，则遍历链表，找到合适的位置将新元素插入。

创建一个新的 `DLL_Element` 对象，并根据排序键值 `sortKey` 设置其成员变量。

更新链表的头部或尾部指针以及相关的链接，将新元素插入到合适的位置。

(2) `SortedRemove` 方法 `void *SortedRemove(int sortKey)`

检查链表是否为空。

如果链表为空，则返回 `NULL`，表示链表中没有元素可以移除。

如果链表不为空，则遍历链表，找到第一个排序键值与 `sortKey` 相等的元素。

如果找到匹配的元素，则将其从链表中移除，并更新链表的头部和尾部指针以及相关的链接。

返回被移除元素的指针。如果没有找到匹配的元素，则返回 `NULL`。

(3) `Prepend` 方法 `void Prepend(void *item)`

检查链表是否为空。

如果链表为空，则创建一个新的 `DLL_Element` 对象，并将其设置为链表的头部和尾部。

如果链表不为空，则创建一个新的 `DLL_Element` 对象，将其设置为链表的头部，并更新链表的头部指针和相关的链接。

(4) `Append` 方法 `void Append(void *item)`

检查链表是否为空。

如果链表为空，则创建一个新的 `DLL_Element` 对象，并将其设置为链表的头部和尾部。

如果链表不为空，则创建一个新的 `DLL_Element` 对象，将其设置为链表的尾部，并更新链表的尾部指针和相关的链接。

(5) `Remove` 方法 `void *Remove(int *keyPtr)`

检查链表是否为空。

如果链表为空，则返回 `NULL`，表示链表中没有元素可以删除。

如果链表不为空，则将链表的头部元素移除，并将其排序键值存储到 `keyPtr` 指针指向的位置。更新链表的头部指针和相关的链接，返回被移除元素的指针。

需要注意的一点是，此处的各个函数之前存在相互调用的关系，其中 `SortedRemove` 的实现用到了 `Remove` 函数和 `IsEmpty` 函数，这也证明了模块化编程的好处：方便代码复用。

3.4.3 测试

我们编译后运行代码进行测试：

```

[ai22920212204180@mc core threads]$ ll
总计 752
-rw-r--r-- 1 ai22920212204180 caodonglin 91 11-02 11:52 bool.h
-rw-r--r-- 1 ai22920212204180 caodonglin 1229 11-02 11:52 copyright.h
-rw-r--r-- 1 ai22920212204180 caodonglin 7200 11-02 13:01 dllist.cc
-rw-r--r-- 1 ai22920212204180 caodonglin 904 11-02 13:01 dllist-driver.cc
-rw-r--r-- 1 ai22920212204180 caodonglin 6236 11-02 13:01 dllist-driver.o
-rw-r--r-- 1 ai22920212204180 caodonglin 909 11-02 11:56 dllist.h
-rw-r--r-- 1 ai22920212204180 caodonglin 45672 11-02 13:01 dllist.o
-rw-r--r-- 1 ai22920212204180 caodonglin 17208 11-02 11:56 interrupt.o
-rw-r--r-- 1 ai22920212204180 caodonglin 7143 11-02 11:52 list.cc
-rw-r--r-- 1 ai22920212204180 caodonglin 2298 11-02 11:52 list.h
-rw-r--r-- 1 ai22920212204180 caodonglin 7796 11-02 11:56 list.o
-rw-r--r-- 1 ai22920212204180 caodonglin 5622 11-02 11:56 main.cc
-rw-r--r-- 1 ai22920212204180 caodonglin 5208 11-02 11:56 main.o
-rw-r--r-- 1 ai22920212204180 caodonglin 23212 11-02 13:01 Makefile
-rwxr-xr-x 1 ai22920212204180 caodonglin 143924 11-02 13:01 nachos
-rw-r--r-- 1 ai22920212204180 caodonglin 5115 11-02 11:52 scheduler.cc
-rw-r--r-- 1 ai22920212204180 caodonglin 1215 11-02 11:52 scheduler.h

```

图 5：dllist-driver.cc 函数

其中三个参数的含义分别在 main.cc 中定义，新的 main.cc 见附录代码五，参数如下：

```

#ifdef THREADS
int testnum;
int thread;
int nodenum;
int error_type;
#endif

```

图 6：参数定义

使用三个线程进行测试：

```

[ai22920212204180@mc core threads]$ ./nachos -m 2 -t 3 -n 2
Thread 0: the number: 383 has entered into the queue
Thread 0: the number: 886 has entered into the queue
Thread 0: the number: 383 has left the queue
Thread 0: the number: 886 has left the queue
The thread 0 : it has been empty ===
Thread 1: the number: 777 has entered into the queue
Thread 1: the number: 915 has entered into the queue
Thread 1: the number: 777 has left the queue
Thread 1: the number: 915 has left the queue
The thread 1 : it has been empty ===
Thread 2: the number: 793 has entered into the queue
Thread 2: the number: 335 has entered into the queue
Thread 2: the number: 335 has left the queue
Thread 2: the number: 793 has left the queue
The thread 2 : it has been empty ===

No threads ready or runnable, and no pending interrupts.
Assuming the program completed.
Machine halting!

Ticks: total 70, idle 0, system 70, user 0
Disk I/O: reads 0, writes 0
Console I/O: reads 0, writes 0
Paging: faults 0
Network I/O: packets received 0, sent 0
Cleaning up...

```

图 7：测试结果

可见，程序正常运行，同时得出结论：正常情况下，前一个线程先插入结点再删除结点，直到链表为空后，下一个线程再执行相同的操作。也即：正常情况下多个线程对链表的操作应该是线性的，总是一个线程全部插入删除完才由另一个线程进行插入和删除。

3.5 插入 yield 语句后，观察并发程序带来的错误

3.5.1 Yield 函数和 Sleep 函数

在开始之前，先了解 Yield 函数的作用，我们在 thread.cc 文件中可以看到 以下内容：

```

// Routines to manage threads.      There are four main operations:

```

```
// Fork -- create a thread to run a procedure concurrently
//      with the caller (this is done in two steps  -- first
//      allocate the Thread object, then call Fork on it)
// Finish -- called when the forked procedure finishes, to clean up
// Yield  -- relinquish control over the CPU to another ready thread
// Sleep  -- relinquish control over the CPU, but thread is now blocked.
//      In other words, it will not run again, until explicitly
//      put back on the ready queue.
```

结合相关资料明白：

yield 方法作用是：暂停当前正在执行的线程对象（及放弃当前拥有的 CPU 资源），并执行其他线程。

yield 做的是让当前运行线程回到可运行状态，以允许具有相同优先级的其他线程获得运行机会。因此，使用 yield() 的目的是让相同优先级的线程之间能适当的轮转执行。yield() 从未导致线程转到等待/睡眠/阻塞状态。在大多数情况下，yield 将导致线程从运行态转到就绪态。而另一个方法 Sleep 会将该线程直接阻塞，在睡眠时间结束后转为就绪态，本次实验无需用到阻塞态。

3.4.2 演示并发错误，并对错误进行归类

这部分是整个实验的核心部分，为了演示可能出现的多种并发错误，我将可能的错误直接写在 Dllist 的实现函数 dllist.cc 中，见附录

(1) main.cc:

```
[ai22920212204180@mc core threads]$ ll
总计 752
-rw-r--r-- 1 ai22920212204180 caodonglin 91 11-02 11:52 bool.h
-rw-r--r-- 1 ai22920212204180 caodonglin 1229 11-02 11:52 copyright.h
-rw-r--r-- 1 ai22920212204180 caodonglin 7380 11-02 13:01 dllist.cc
-rw-r--r-- 1 ai22920212204180 caodonglin 904 11-02 13:01 dllist-driver.cc
-rw-r--r-- 1 ai22920212204180 caodonglin 6236 11-02 13:01 dllist-driver.o
-rw-r--r-- 1 ai22920212204180 caodonglin 909 11-02 11:56 dllist.h
-rw-r--r-- 1 ai22920212204180 caodonglin 45672 11-02 13:01 dllist.o
-rw-r--r-- 1 ai22920212204180 caodonglin 17208 11-02 11:56 interrupt.o
-rw-r--r-- 1 ai22920212204180 caodonglin 7143 11-02 11:52 list.cc
-rw-r--r-- 1 ai22920212204180 caodonglin 2298 11-02 11:52 list.h
-rw-r--r-- 1 ai22920212204180 caodonglin 7796 11-02 11:56 list.o
-rw-r--r-- 1 ai22920212204180 caodonglin 5622 11-02 11:56 main.cc
-rw-r--r-- 1 ai22920212204180 caodonglin 5208 11-02 11:56 main.o
-rw-r--r-- 1 ai22920212204180 caodonglin 23212 11-02 13:01 Makefile
-rwxr-xr-x 1 ai22920212204180 caodonglin 143924 11-02 13:01 nachos
-rw-r--r-- 1 ai22920212204180 caodonglin 5115 11-02 11:52 scheduler.cc
-rw-r--r-- 1 ai22920212204180 caodonglin 1215 11-02 11:52 scheduler.h
-rw-r--r-- 1 ai22920212204180 caodonglin 10004 11-02 11:56 scheduler.o
-rw-r--r-- 1 ai22920212204180 caodonglin 4304 11-02 11:56 stats.o
-rw-r--r-- 1 ai22920212204180 caodonglin 5794 11-02 11:52 stdarg.h
-rw-r--r-- 1 ai22920212204180 caodonglin 9083 11-02 11:57 switch.c
-rw-r--r-- 1 ai22920212204180 caodonglin 3806 11-02 11:52 switch.h
```

图 8：main.cc

(2) 对于 threadtest.cc，在演示并发程序时只用到 ThreadTest2 函数，程序在 main 函数中调用 ThreadTest 方法，在 ThreadTest 方法中通过参数 m==2 确定当前程序是并发的，随后跳转到 ThreadTest2 中，ThreadTest2 方法通过调用 TestDllist 方法来产生按照序号命名的线程，线程中对应着不同的双向链表。新的 threadtest.cc 见附录六，其调用关系如下：

```

> void SimpleThread(int which) ...
// test are threads all empty
> void TestDlList(int which) ...
//-----
// ThreadTest1
// Set up a ping-pong between two threads, by forking a thread
// to call SimpleThread and then calling SimpleThread ourselves.
//-----
> void ThreadTest1() ...
//-----
// ThreadTest2
//-----
> void ThreadTest2() ...
//-----
// ThreadTest
// Invoke a test routine.
//-----
> void ThreadTest() ...

```

图 9：threadtest.cc

- (3) 使用命令行参数，在 dlList.cc 的适当位置添加 Yield 语句，观察程序出现的并发错误，这部分的代码详见前文所述，为了捕捉到所有可能出现的错误，我采用白盒测试中的语句覆盖方法，在可能出现问题代码前都加入 Yield 语句，然后通过命令行参数决定当前发生哪个错误可用。命令格式如下，其中 m 代表模式，为 1 时调用 nachos 样例中的两个线程；为 2 时调用我写的双向链表对应的线程，t 代表线程数量，n 代表每个线程的结点个数，e 代表错误编号，不输入则代表不展示错误：

```

[ai22920212204180@mc core threads]$ ./nachos -m 2 -t 2 -n 2 -e 2

```

图 10：命令行参数示意

- (4) 错误展示：

a) 在初始化时切换线程：

```

[ai22920212204180@mc core threads]$ ./nachos -m 2 -t 2 -n 2 -e 1
Error 1
Error 1
Thread 0: the number: 383 has entered into the queue
Error 1
Thread 1: the number: 886 has entered into the queue
Error 1
Thread 0: the number: 777 has entered into the queue
Thread 0: the number: 777 has left the queue
Thread 0: the number: 886 has left the queue
The thread 0 :it has been empty ==
段错误

```

图 11：错误 1

注意到本应在线程 1 中的元素 886 进入了线程 0，本该在线程 0 中的 383 却消失了，这是因为线程 0 在初始化未完成时就切换到了线程 1，导致应该插入到首位的结点游离在外。

```

DLLElement::DLLElement(void *itemPtr, int sortKey)
{
    // <- error 1 --
    if (error_type == 1)
    {
        printf("Error 1\n");
        currentThread->Yield();
    }
    // -- error 1 ->
    item = itemPtr;
    key = sortKey;
    prev = NULL;
    next = NULL;
}

```

图 12: 错误 1 对应代码

b) 在移除一个元素时切换线程，观察下图：

```

[ai22920212204180@mc core threads]$ ./nachos -m 2 -t 2 -n 2 -e 2
Thread 0: the number: 383 has entered into the queue
Thread 0: the number: 886 has entered into the queue
Error 2
Thread 1: the number: 777 has entered into the queue
Thread 1: the number: 915 has entered into the queue
Error 2
Thread 0: the number: 383 has left the queue
Error 2
Thread 1: the number: 777 has left the queue
Thread 1: the number: 915 has left the queue
The thread 1 :it has been empty ===

段错误
[ai22920212204180@mc core threads]$ █

```

图 13: 错误 2

当要移除线程 0 中 886 元素时，首先移动 first 指针，然后把原来的首节点移去，但此时线程发生了切换，等切换回来的时候 first 已经改变，不能成功移去首结点：

```

*keyPtr = first->key;
RemovedItem = &(first->key);
first = first->next;
if (first == NULL)
    last = NULL;
else
{
    // <- error 2 --
    if (error_type == 2)
    {
        printf("Error 2\n");
        currentThread->Yield();
    }
    // -- error 2 ->
    first->prev = NULL;
}

```

图 14: 错误 2 对应代码

- c) 在有序地插入一个空链表时切换线程，观察下图：

```
段错误
[ai22920212204180@mc core threads]$ ./nachos -m 2 -t 2 -n 2 -e 3
Error 3
段错误
[ai22920212204180@mc core threads]$
```

图 15: 错误 3

在空链表中正准备插入时发生切换，此时已经修改了 first 指针，但还没有修改 last 指针，等切换回来的时候已经不能修改 last 指针，从而发生段错误：

```
DLLElement *temp = new DLLElement(item, sortKey);
temp->prev = NULL;
temp->next = NULL;
first = temp;
// <- error 3 --
if (error_type == 3)
{
    printf("Error 3\n");
    currentThread->Yield();
}
last = temp;
// -- error 3 ->
```

图 16: 错误 3 对应代码

- d) 在插入一个非空链表时：

```
[ai22920212204180@mc core threads]$ ./nachos -m 2 -t 2 -n 2 -e 4
Thread 0: the number: 383 has entered into the queue
Error 4
Error 4
Thread 0: the number: 886 has entered into the queue
Thread 0: the number: 383 has left the queue
Thread 0: the number: 886 has left the queue
The thread 0 :it has been empty ==
段错误
[ai22920212204180@mc core threads]$
```

图 17: 错误 4

你需要遍历找到插入位置，在遍历之初切换线程，将会导致无法正常遍历，从而如图所示，无法启动第二个线程：

```
DLLElement *temp = new DLLElement(item, sortKey);
DLLElement *traverse = first;
// <- error 4 --
if (error_type == 4)
{
    printf("Error 4\n");
    currentThread->Yield();
}
// -- error 4 ->
```

图 18: 错误 4 对应代码

- e) 在插入一个非空链表时，你需要遍历找到插入位置，在遍历过程中切换线程，当遍历开始进行时切换线程：

```

段错误
[ai22920212204180@mc core threads]$ ./nachos -m 2 -t 2 -n 2 -e 5
Thread 0: the number: 383 has entered into the queue
Error 5
Error 5
Thread 0: the number: 886 has entered into the queue
Thread 0: the number: 383 has left the queue
Thread 0: the number: 886 has left the queue
The thread 0 :it has been empty ===

段错误
[ai22920212204180@mc core threads]$ █

```

图 19: 错误 5

如图，当切换回当前线程时，无法继续正常遍历：

```

while (traverse != NULL && sortKey >= traverse->key)
{
    // <- error 5 --
    if (error_type == 5)
    {
        printf("Error 5\n");
        currentThread->Yield();
    }
    // -- error 5 ->
    traverse = traverse->next;
}

```

图 20: 错误 5 及错误对应代码

f) 插入非空链表的尾部时切换线程：

```

段错误
[ai22920212204180@mc core threads]$ ./nachos -m 2 -t 2 -n 2 -e 6
Thread 0: the number: 383 has entered into the queue
Error 6
Thread 1: the number: 777 has entered into the queue
Error 6
Thread 0: the number: 886 has entered into the queue
Thread 0: the number: 383 has left the queue
Thread 0: the number: 915 has left the queue
The thread 0 :it has been empty ===

Thread 1: the number: 915 has entered into the queue
段错误
[ai22920212204180@mc core threads]$ █

```

图 21: 错误 6

修改完链表以后需要修改 last 指针，若在修改 last 指针之前就切换线程，会导致无法正常修改 last 指针。

```

temp->prev = last;
temp->next = NULL;
last->next = temp;
// <- error 6 --
if (error_type == 6)
{
    printf("Error 6\n");
    currentThread->Yield();
}
// -- error 6 ->
last = temp;

```

图 22: 错误 6 及错误对应代码

g) 插入非空链表的头部时切换线程:

```

[ai22920212204180@mcore threads]$ ./nachos -m 2 -t 3 -n 4 -e 7
Thread 0: the number: 383 has entered into the queue
Thread 0: the number: 886 has entered into the queue
Thread 0: the number: 777 has entered into the queue
Thread 0: the number: 915 has entered into the queue
Thread 0: the number: 383 has left the queue
Thread 0: the number: 777 has left the queue
Thread 0: the number: 886 has left the queue
Thread 0: the number: 915 has left the queue
The thread 0 :it has been empty ===

Thread 1: the number: 793 has entered into the queue
Error 7
Error 7
Thread 1: the number: 335 has entered into the queue
Thread 1: the number: 492 has entered into the queue
Thread 1: the number: 649 has entered into the queue
Thread 1: the number: 335 has left the queue
Thread 1: the number: 492 has left the queue
Thread 1: the number: 649 has left the queue
Thread 1: the number: 793 has left the queue
The thread 1 :it has been empty ===

段错误
[ai22920212204180@mcore threads]$ █

```

图 23: 错误 7

在新的头节点已经做好准备, 指针指向正确, 但还没有放入链表时切换线程, 会导致新的头节点无法成功放入链表, 从而导致错误:

```

temp->prev = NULL;
temp->next = first;
// <- error 7 --
if (error_type == 7)
{
    printf("Error 7\n");
    currentThread->Yield();
}
// -- error 7 ->

```

图 24: 错误 7 对应代码

h) 插入非空链表的头部时, 新的头结点已经进入链表, 但头节点还没有切换, 此时切换线程, 将会导致新的 first 不正确:

```

[ai22920212204180@mcore threads]$ ./nachos -m 2 -t 3 -n 4 -e 8
Thread 0: the number: 383 has entered into the queue
Thread 0: the number: 886 has entered into the queue
Thread 0: the number: 777 has entered into the queue
Thread 0: the number: 915 has entered into the queue
Thread 0: the number: 383 has left the queue
Thread 0: the number: 777 has left the queue
Thread 0: the number: 886 has left the queue
Thread 0: the number: 915 has left the queue
The thread 0 :it has been empty ===

Thread 1: the number: 793 has entered into the queue
Error 8
Error 8
Thread 1: the number: 335 has entered into the queue
Thread 1: the number: 492 has entered into the queue
Thread 1: the number: 649 has entered into the queue
Thread 1: the number: 335 has left the queue
Thread 1: the number: 793 has left the queue
段错误

```

图 25: 错误 8

```

// <- error 8 --
if (error_type == 8)
{
    printf("Error 8\n");
    currentThread->Yield();
}
// -- error 8 ->
first = temp;

```

图 26: 错误 8 对应代码

- i) 在链表中间插入时，放入新节点前切换线程：

```

[ai22920212204180@mc core threads]$ ./nachos -m 2 -t 3 -n 4 -e 9
Thread 0: the number: 383 has entered into the queue
Thread 0: the number: 886 has entered into the queue
Error 9
Thread 1: the number: 915 has entered into the queue
Error 9
Thread 2: the number: 335 has entered into the queue
Error 9
Thread 0: the number: 777 has entered into the queue
Error 9
Thread 1: the number: 793 has entered into the queue
Error 9
Thread 2: the number: 386 has entered into the queue
Error 9
Thread 0: the number: 492 has entered into the queue
Thread 0: the number: 335 has left the queue
Thread 0: the number: 383 has left the queue
Thread 0: the number: 492 has left the queue
Thread 0: the number: 777 has left the queue
The thread 0 :it hasn't been empty ===
段错误

```

图 27: 错误 9

此时新节点还没有成功进入链表，切换线程会导致新的节点无法成功放入链表，从而导致错误

```

// <- error 9 --
if (error_type == 9)
{
    printf("Error 9\n");
    currentThread->Yield();
}
temp->prev = traverse->prev;
// -- error 9 ->

```

图 28: 错误 9 对应代码

- j) 在链表中间插入时，放入新节点过程中切换线程：

```

[ai22920212204180@mc core threads]$ ./nachos -m 2 -t 3 -n 4 -e 10
Thread 0: the number: 383 has entered into the queue
Thread 0: the number: 886 has entered into the queue
Error 10
Thread 1: the number: 915 has entered into the queue
Thread 1: the number: 793 has entered into the queue
Thread 1: the number: 335 has entered into the queue
Error 10
Thread 2: the number: 492 has entered into the queue
Thread 2: the number: 649 has entered into the queue
Thread 2: the number: 421 has entered into the queue
Error 10
Thread 0: the number: 777 has entered into the queue
Thread 0: the number: 27 has entered into the queue
Thread 0: the number: 27 has left the queue
Thread 0: the number: 335 has left the queue
Thread 0: the number: 362 has left the queue
段错误

```

图 29: 错误 10

要理解错误 10, 就要先理解在链表中插入结点的过程:

以下是插入最初状态, traverse 代表要插入的那个点之前的位置:

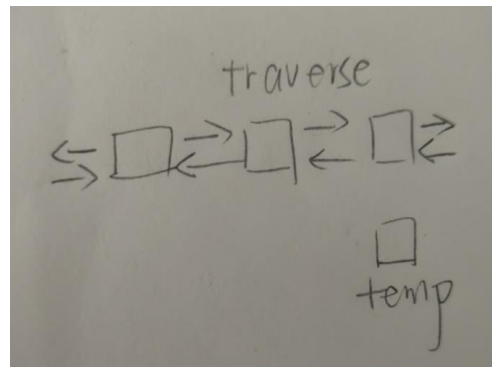


图 30: 插入初始状态

要成功插入就要经过以下四个步骤:

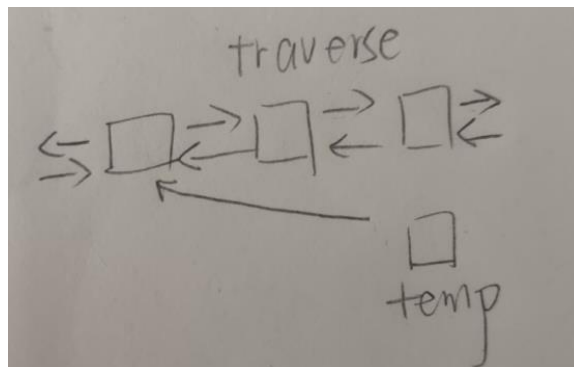


图 31: 插入第一步

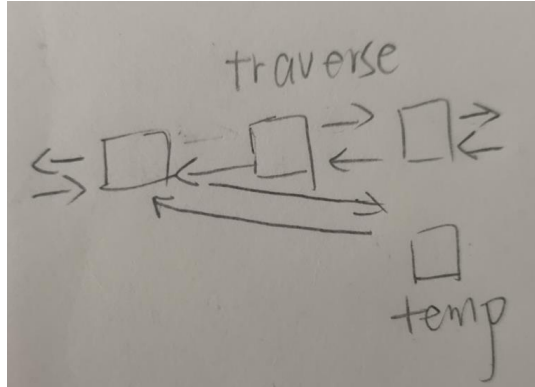


图 32: 插入第二步

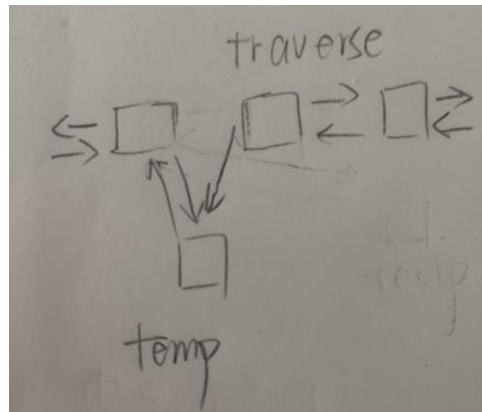


图 33: 插入第三步

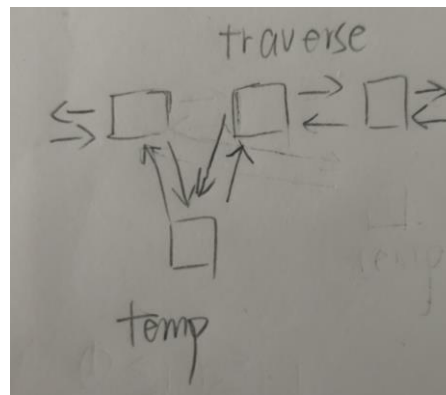


图 34: 插入第四步

在完成第二步，但还没有完成第三步时切换线程，将会导致第三步无法成功进行，从而导致错误。

```
// error 10
traverse->prev->next = temp;
// <- error 10 --
if (error_type == 10)
{
    printf("Error 10\n");
    currentThread->yield();
}
traverse->prev = temp;
// -- error 10 ->
temp->next = traverse;
```

图 35: 错误 10 对应代码

(5) 错误归纳与总结:

从线程切换的时间上说:

错误 1 发生于析构函数;

错误 2 发生于 remove 函数, 删除结点的实现过程中;

错误 3-10 发生于具体的结点删除过程中, 其中又分为删除头节点、删除非头节点、于不同时刻删除结点等具体状况;

从性质上说:

所有错误均是由于切换线程后指针指向发生改变导致, 只有由没做好线程管理, 共享资源混用导致的, 由此也能看出多线程的实现多么困难、操作系统多么强大。

4 实验总结

4.1 通过环境的配置和前期的准备工作

我认识了 nachos 这个用软件模型的操作系统平台, 学会了从零开始看一个操作系统的源代码, 理解线程部分的执行和实现。

4.2 通过对 thread 目录下各种文件的阅读

我巩固了 C++ 多文件编程的方式, 进一步搞懂了使用宏这种方法来避免头文件重复引入的方法, 同时, 进一步规范了自己的代码书写结构。

4.3 学会了更加规范的注释方法

块注释通常用于方法之前, 用来解释整个方法的作用。在块注释中, 一般会包含方法的参数、输入和输出等详细说明, 以帮助其他开发人员理解该方法的功能。块注释的主要目的是提供全面的文档, 使得项目易于理解和测试。

而行注释则更多地用于方法内部, 用来解释某句代码的含义或者说明重要的、难以理解的代码段。行注释通常是针对具体的代码行或者代码块的解释, 通过这种注释, 开发人员可以更容易地理解和维护代码。行注释的另一个优点是, 相关的软件工具可以直接根据这些注释生成代码的开发文档, 进一步提高代码的可读性和可维护性。

综上所述, 块注释常用于方法前面, 用来解释整个方法的作用、参数、输入和输出等信息, 而行注释主要用于方法内部, 解释具体代码行或代码块的含义, 便于理解和测试, 同时也方便相关工具生成代码文档。这两种注释形式的使用有助于提高代码的可读性和可维护性。

4.4 通过对并发程序部分的实验

加深了对线程调度、程序并发和互斥等概念的理解。我了解到线程调度在多线程程序中的重要性, 以及错误的线程调度可能导致的问题。我还注意到在特定情况下使用线程调度函数是有害的, 因为它可能导致不正确的线程切换。这进一步强调了正确使用线程调度的必要性, 以确保程序的正确执行。

此外, 我还意识到信号量的有用之处。信号量是一种用于控制并发访问共享资源的工具。通过使用信号量, 可以实现线程之间的同步和互斥, 从而避免竞态条件和数据不一致性的问题。信号量可以用于限制同时访问共享资源的线程数量, 确保只有一个线程访问该资源, 或者用于控制线程的执行顺序, 以实现特定的同步要求。

4.5 通过对 makefile 和 gcc 的对比学习

我巩固了编译的相关知识, 特别是通过本次的多文件编写项目, 深刻地认识到了“链接”这个步骤的重要性。

5 实验心得

本次实验总体来讲，主要问题在于 make 过程总是不成功，在我反复操作无果，被迫学习了与编译相关的各种知识依然不能解决问题，一怒之下删除所有文件重做，然后终于成功了，但最终也不知道当初为什么出错。

在代码实现过程中，使用纸笔模拟对链表的操作是十分有用且高效的手段，在多线程并行下有助于生动形象地理解并发的过程。

6 附录

代码三 dlist.cc:

```
DLLElement::DLLElement(void *itemPtr, int sortKey)
{
    if (error_type == 1)
    {
        printf("Error 1\n");
        currentThread->Yield();
    }
    item = itemPtr;
    key = sortKey;
    prev = NULL;
    next = NULL;
}

DLLList::DLLList()
{
    first = NULL;
    last = NULL;
}

DLLList::~~DLLList()
{
    while (this->IsEmpty())
    {
        int k;
        this->Remove(&k);
    }
}

void DLLList::Prepend(void *item)
{
    if (!this->IsEmpty())
    {
        DLLElement *temp = new DLLElement(item, START_KEY);
        temp->prev = NULL;
        temp->next = NULL;
```

```

        first = temp;
        last = temp;
    }
    else
    {
        DLLElement *temp = new DLLElement(item, first->key - 1);
        temp->prev = NULL;
        temp->next = first;
        first->prev = temp;
        first = temp;
    }
}

void DLLList::Append(void *item)
{
    if (!this->IsEmpty())
    {
        DLLElement *temp = new DLLElement(item, START_KEY);
        temp->prev = NULL;
        temp->next = NULL;
        first = temp;
        last = temp;
    }
    else
    {
        DLLElement *temp = new DLLElement(item, last->key + 1);
        temp->prev = last;
        temp->next = NULL;
        last->next = temp;
        last = temp;
    }
}

void *DLLList::Remove(int *keyPtr)
{
    void *RemovedItem;
    if (!this->IsEmpty())
    {
        keyPtr = NULL;
        return NULL;
    }
    else
    {
        *keyPtr = first->key;
    }
}

```

```

        RemovedItem = &(first->key);
        first = first->next;
        if (first == NULL)
            last = NULL;
        else
        {
            if (error_type == 2)
            {
                printf("Error 2\n");
                currentThread->Yield();
            }

            first->prev = NULL;
        }
    }
    return RemovedItem;
}

bool DLLList::IsEmpty()
{
    if (first == NULL)
        return false;
    else
        return true;
}

void DLLList::SortedInsert(void *item, int sortKey)
{
    if (!this->IsEmpty())
    {
        DLLElement *temp = new DLLElement(item, sortKey);
        temp->prev = NULL;
        temp->next = NULL;
        first = temp;
        if (error_type == 3)
        {
            printf("Error 3\n");
            currentThread->Yield();
        }
        last = temp;
    }
    else
    {
        DLLElement *temp = new DLLElement(item, sortKey);
        DLLElement *traverse = first;
    }
}

```

```

        if (error_type == 4)
        {
            printf("Error 4\n");
            currentThread->Yield();
        }

        while (traverse != NULL && sortKey >= traverse->key)
        {
            if (error_type == 5)
            {
                printf("Error 5\n");
                currentThread->Yield();
            }

            traverse = traverse->next;
        }

        if (traverse == NULL)
        {
            temp->prev = last;
            temp->next = NULL;
            last->next = temp;
            if (error_type == 6)
            {
                printf("Error 6\n");
                currentThread->Yield();
            }

            last = temp;
        }

        else if (traverse == first)
        {
            temp->prev = NULL;
            temp->next = first;
            if (error_type == 7)
            {
                printf("Error 7\n");
                currentThread->Yield();
            }

            first->prev = temp;
            if (error_type == 8)
            {
                printf("Error 8\n");
                currentThread->Yield();
            }

            first = temp;
        }

        else

```

```

        {
            if (error_type == 9)
            {
                printf("Error 9\n");
                currentThread->Yield();
            }
            temp->prev = traverse->prev;
            traverse->prev->next = temp;
            if (error_type == 10)
            {
                printf("Error 10\n");
                currentThread->Yield();
            }
            traverse->prev = temp;
            temp->next = traverse;
        }
    }
}

void *DLLList::SortedRemove(int sortKey)
{
    void *ReturnItem;
    if (!this->IsEmpty())
        return NULL;
    else
    {
        DLLElement *traverse = first;
        while (traverse != NULL && sortKey != traverse->key)
            traverse = traverse->next;
        if (traverse == NULL)
            return NULL;
        else
        {
            ReturnItem = traverse->item;
            if (traverse == first)
            {
                int *key;
                this->Remove(key);
            }
            else if (traverse == last)
            {
                last = last->prev;
                last->next = NULL;
                if (last == NULL)

```

```

        first = NULL;
    }
    else
    {
        traverse->prev->next = traverse->next;
        traverse->next->prev = traverse->prev;
    }
    return ReturnItem;
}
}
}

```

代码四 dllist-driver.cc

```

void InsertList(DLList *list, int N)
{
    for (int i = 0; i < N; i++)
    {
        void *item;          int key = Random() % 1000;
        int *items = new int[1];
        items[0] = key;
        list->SortedInsert(item, key);
        printf("%s: the number: %d has entered into the queue \n",
            currentThread->getName(), key);
    }
}

void RemoveList(DLList *list, int N)
{
    for (int i = 0; i < N; i++)
    {
        int key;
        void *item;
        item = list->Remove(&key);
        printf("%s: the number: %d has left the queue \n",
            currentThread->getName(),
            *((int *)item));
    }
}

```

代码五 main.cc

```

#ifdef THREADS
    for (argc--, argv++; argc > 0; argc -= argcCount, argv +=

```

```

                                                                    argCount)
{
    argCount = 1;
    switch (argv[0][1])
    {
        case 'm':            testnum = atoi(argv[1]);
                            argCount++;
                            break;
        case 't':            thread = atoi(argv[1]);
                            argCount++;
                            break;
        case 'n':            nodenum = atoi(argv[1]);
                            argCount++;
                            break;
        case 'e':            error_type = atoi(argv[1]);
                            argCount++;
                            break;
        default:              testnum = 1;
                            break;
    }
}
ThreadTest();
#endif

```

代码六 threadtest.cc

```

void SimpleThread(int which)
{
    int num;

    for (num = 0; num < 5; num++)
    {
        printf("*** thread %d looped %d times\n", which, num);
        currentThread->Yield();
    }
}

void TestDllist(int which)
{
    InsertList(list, nodenum);
    RemoveList(list, nodenum);
    if (!list->IsEmpty())
        printf("The thread %d :it has been empty ==\n\n", which);
    else

```

```

        printf("The thread %d :it hasn't been empty ==\n\n", which);
    }

void ThreadTest1()
{
    DEBUG('t', "Entering ThreadTest1");

    Thread *t = new Thread("forked thread");

    t->Fork(SimpleThread, 1);
    SimpleThread(0);
}

void ThreadTest2()
{
    DEBUG('t', "Entering Multi-thread-test ");
    list = new DLLlist();
    for (int i = 0; i < thread; i++)
    {
        char No[4] = "1";
        sprintf(No, "%d", i);
        char *name = new char[25];
        name[0] = '\0';
        strcat(name, "Thread ");
        strcat(name, No);
        Thread *t = new Thread(name);
        t->Fork(TestDllist, i);
    }
}

void ThreadTest()
{
    switch (testnum)
    {
    case 1:
        ThreadTest1();
        break;
    case 2:
        ThreadTest2();
        break;
    default:
        printf("No test specified.\n");
    }
}

```

```
        break;  
    }  
}
```